Solution: Minimum Window Substring

Problem Statement

Given a string `s` and `t` of length `m` and `n` respectively, find the `smallest substring` in a given string 's' that contains all the characters (including duplicate) of another string 't'. The `order of characters` in 't' doesn't matter, but the substring in 's' must include all characters of 't', possibly with additional characters.

If no such substring exists, return an `empty` string.

Examples

Example 1:

- **Input**: s = "xyyzyzyx", t = "xyz"
- **Expected Output**: "zyx"
- **Justification**: "zyx" is the shortest substring of 's' that contains 'x', 'y', and 'z'.

Example 2:

- **Input**: s = "abracadabra", t = "aa"
- **Expected Output**: "ada"
- **Justification**: "ada" at the end of 's' is the shortest substring containing all 'a's in 't'.

Example 3:

- **Input**: s = "welcomeback", t = "ole"
- **Expected Output**: "elco"
- **Justification**: "elco" includes 'o', 'l', and 'e', fulfilling the requirements with the minimum length.

Solution

To solve this problem, we'll use a sliding window approach combined with a character frequency map. This approach is efficient because it involves scanning the string 's' only once while adjusting the window's start and end to include all characters of 't'.

Initially, we'll create a map to keep track of the frequency of each character in 't'. As we traverse 's', we'll adjust the end of the window to include characters and decrease their frequency in the map. Once all characters from 't' are included in the window, we'll attempt to shrink the window from the start, checking if it still contains all necessary characters. This approach ensures we always have the minimum window containing all characters of 't', which is the core requirement of the problem.

Step-by-step Algorithm

- Initialize a map to store the frequency of each character in 't'.
- Use two pointers, `start` and `end`, to represent the current window in 's'.
- Iterate over 's' using the `end` pointer.
  - For each character in 's' at `end`, decrease its frequency in the map.
  - Once all characters from 't' are included in the current window (checked via the map), try to shrink the window.
    - Move the `start` pointer forward while the window still contains all characters from 't'.
  - Update the minimum window size and its starting index when a smaller valid window is found.
- Return the smallest valid substring from 's' based on the recorded starting index and size. If no such window exists, return an empty string.

Algorithm Walkthrough

Let's take "xyyzyzyx" as 's' and "xyz" as 't'.

Initialization:

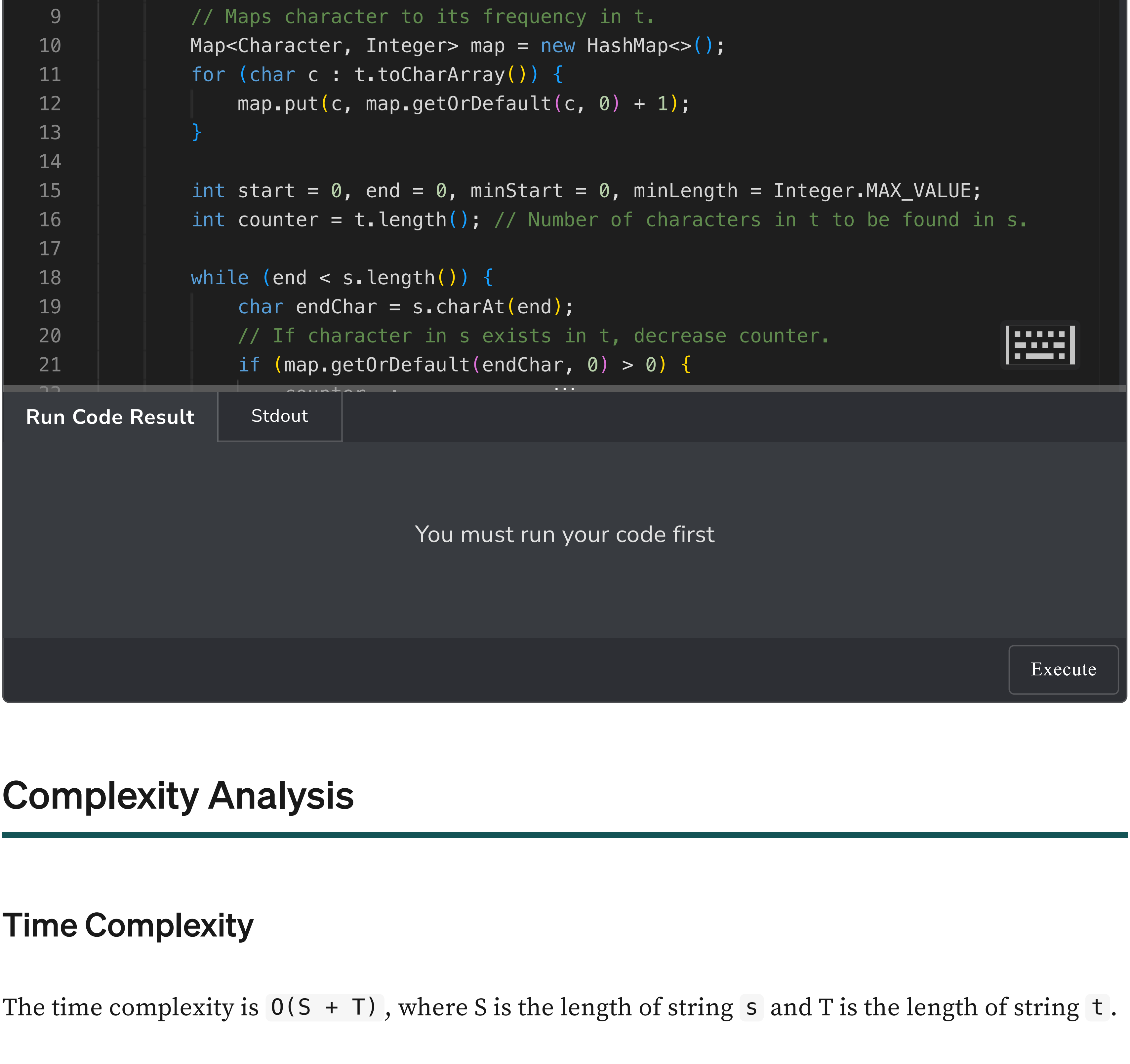
- **Map Creation**: Create a frequency map for characters in `t`. Map: {x: 1, y: 1, z: 1}.
- **Variable Initialization**: Set `start` and `end` to 0, `minStart` to 0, `minLength` to a large value (like Integer.MAX_VALUE), and `counter` to the length of `t`, which is 3.

1. **First Expansion**:
   - `end` = 0, `s[end]` = 'x'. Map: {x: 0, y: 1, z: 1}, `counter` = 2. Window: "x".
2. **Second Expansion**:
   - `end` = 1, `s[end]` = 'y'. Map: {x: 0, y: 0, z: 1}, `counter` = 1. Window: "xy".
3. **Third Expansion**:
   - `end` = 2, `s[end]` = 'y'. Map: {x: 0, y: -1, z: 1}. Window: "xyy".
4. **Fourth Expansion**:
   - `end` = 3, `s[end]` = 'z'. Map: {x: 0, y: -1, z: 0}, `counter` = 0. Window: "xyyz".
5. **Start Shrinking**:
   - `start` = 1, `s[start - 1]` = 'x'. Map: {x: 1, y: -1, z: 0}. Window: "yyz".
6. **Fifth Expansion**:
   - `end` = 4, `s[end]` = 'y'. Map: {x: 1, y: -2, z: 0}. Window: "yyzy".
7. **Sixth Expansion**:
   - `end` = 5, `s[end]` = 'z'. Map: {x: 1, y: -2, z: -1}. Window: "yyzyz".
8. **Seventh Expansion**:
   - `end` = 6, `s[end]` = 'y'. Map: {x: 1, y: -3, z: -1}. Window: "yyzyzy".
9. **Eighth Expansion**:
   - `end` = 7, `s[end]` = 'x'. Map: {x: 0, y: -3, z: -1}. Window: "yzyzyx".
   - Current window contains all characters of string `t`. `minLength`: min(4, 6) = 4, `minStart`: 2.
   - `start` is now incremented to find a smaller window containing all characters.
10. **Shrinking Again**:
    - **Shrink**: `start=2`, `end=7`, Window: "yzyzyx", Map: {x: 0, y: -2, z: -1}, Counter remains 0.
    - **Shrink**: `start=3`, `end=7`, Window: "zyzyx", Map: {x: 0, y: -1, z: -1}, Counter remains 0.
    - **Shrink**: `start=4`, `end=7`, Window: "yzyx", Map: Map: {x: 0, y: -1, z: 0}, Counter remains 0.
    - **Shrink**: `start=5`, `end=7`, Window: "zyx", Map: {x: 0; y: 0, z: 0}, Counter becomes 1.
    - This is the smallest window that contains all characters of `t`.

Result:

- The minimum window substring that contains all characters of `t` ("xyz") is "zyx".

Code



```java
import java.util.*;

public class Solution {
    public String minWindow(String s, String t) {
        if (s == null || t == null || s.length() == 0 || t.length() == 0) {
            return "";
        }

        // Maps character to its frequency in t.
        Map<Character, Integer> map = new HashMap<>();
        for (char c : t.toCharArray()) {
            map.put(c, map.getOrDefault(c, 0) + 1);
        }

        int start = 0, end = 0, minStart = 0, minLength = Integer.MAX_VALUE;
        int counter = t.length(); // Number of characters in t to be found in s.

        while (end < s.length()) {
            char endChar = s.charAt(end);
            // If character in s exists in t, decrease counter.
            if (map.getOrDefault(endChar, 0) > 0) {
```

```
You must run your code first
```

Complexity Analysis

Time Complexity

The time complexity is `O(S + T)`, where S is the length of string `s` and T is the length of string `t`.

This is because the algorithm iterates through both strings linearly. The outer loop runs for every character in `s`, and the inner while loop processes each character only once when shrinking the window.

Space Complexity

The space complexity is `O(T)`, where T is the length of string `t`.

This complexity arises from the storage required for the frequency map of characters in `t`. In the worst case, this map will store a distinct entry for each character in `t`.

← Previous
Minimum Window Substring (Hard)

Next →
Best Meeting Point (Hard)

☐ Mark as Completed